

İzmir University of Economics

2024-2025 Spring

SE311 Project



Livestock Farm

Group Members:

Begüm Gençer

Cemile Dilvin Ağaçhanlı

Sinem Yeşil

Tuana Kara

1.Introduction

This project is a simulation where cattle on the farm are monitored with electronic devices. This system can be used to track the location of the cattle and warn the farmer when the cattle go out of the farm border. The system uses devices that use Zigbee signals in most cattle and Bluetooth signals in some. The farm also manages the feeding of dairy and meat cattle on the farm, as they have different diets. At the same time, veterinarians check whether the cattle are vaccinated, and the Ministry of Agriculture and Livestock checks whether the cattle have a plastic ear tag that contains a unique ID.

2.Design Patterns Used

2.1 Adapter Pattern

Why Used: The server only accepts Zigbee signals, but some cattle use Bluetooth devices.

Implementation: The Adapter pattern is used to convert Bluetooth signals that the server does not accept to the Zigbee format to be compatible with the server. The BluetoothToZigbeeAdapter class implements the Device interface and allows Bluetooth devices to work and communicate with the server without any problems.

Participants Mapping:

- BluetoothDevice: Adaptee
- ZigbeeDevice: Target interface
- BluetoothToZigbeeAdapter: Adapter

2.2 Observer Pattern

Why Used: When a cattle leaves the specified farm boundaries, the farmer needs to be notified.

Implementation: Each Cattle object checks its location. If it is outside the boundaries, the Farmer is notified by the notifyObservers() method.

Participants Mapping:

- Cattle: Subject

- Farmer: Observer

2.3 Singleton Pattern

Why Used: Ensuring that all cattle use a common, single location database and preventing external access.

Implementation:

Location Database class is defined as Singleton and only one instance can be created and used in common in the system.

Participants Mapping:

- LocationDatabase: Singleton class

2.4 Abstract Factory Pattern

Why Used: Dairy and beef cattle have different feed types. It was used to manage this situation with a consistent structure.

Implementation: FeedFactory is an abstract class. It has two real (concrete) implementations: DairyFeedFactory and BeefFeedFactory. Each provides carbohydrate and protein feed appropriate for the cattle breed.

Participants Mapping:

- FeedFactory: Abstract Factory
- DairyFeedFactory, BeefFeedFactory: Concrete Factories
- Carbohydrate, Protein: Abstract Products
- Corn, Soybean, Wheat, Canola: Concrete Products

2.5 Visitor Pattern

Why Used: It has been used to perform actions such as vaccination, ear tag check on all cattle without changing the cattle class.

Implementation: There are two classes of visitors: Veterinarian and Ministry Inspector. They visit every cattle farm and check vaccinations and ear tags.

Participants Mapping:

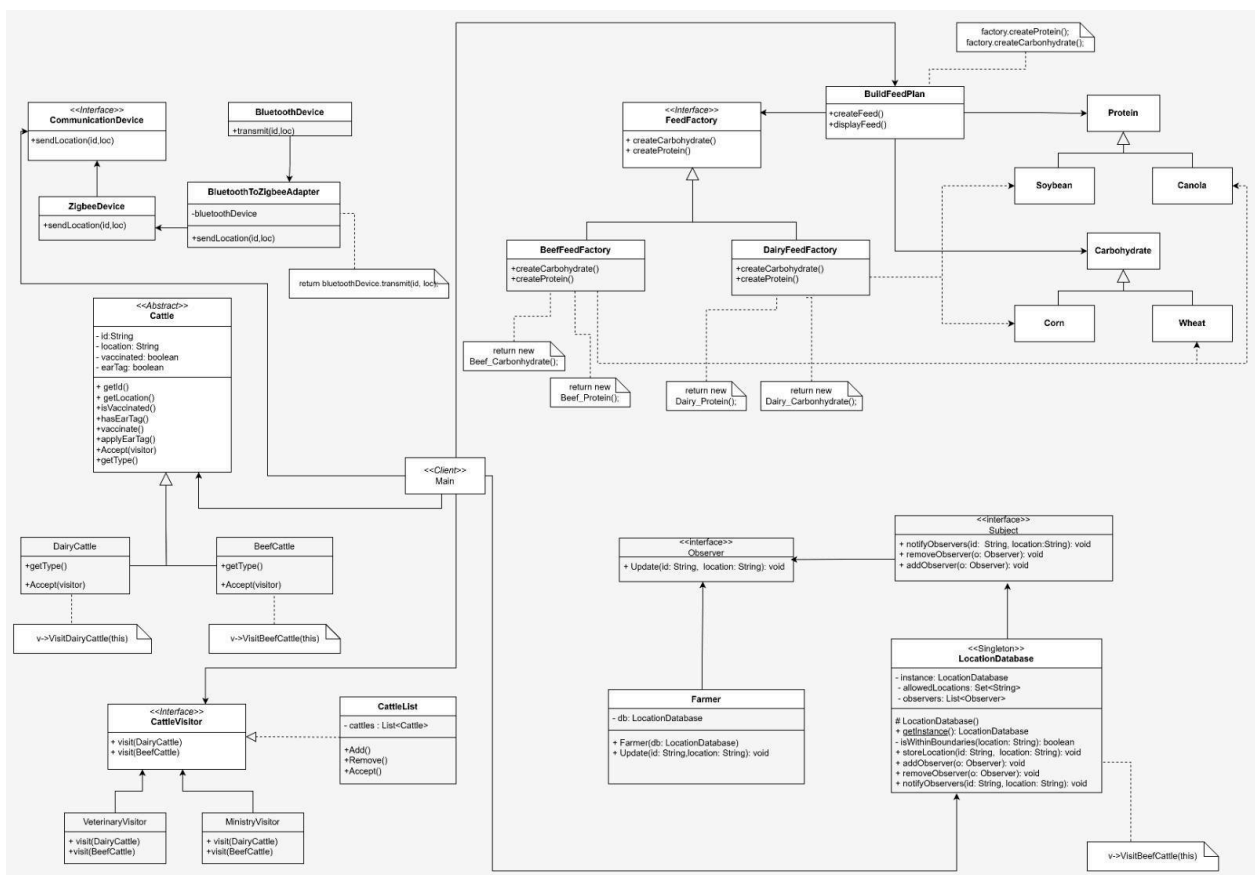
- Cattle: Element
- Veterinarian, MinistryVisitor: Visitors

3.Class Design and Method Explanation

- **Cattle (Abstract Class):** General cattle class. Contains basic information such as ID and location. Accepts visitors.
 - accept(Visitor visitor): Used in the Visitor pattern to accept operations from external classes.
- **DairyCattle / BeefCattle:** They are subclasses derived from the Cattle class. They have different feeding types.
- **FeedFactory (Abstract class):** Provides an interface to create a feed object. Defines two abstract methods.
 - createCarbohydrate(): Returns a carbohydrate object.
 - createProtein(): Returns a protein object.
- **DairyFeedFactory / BeefFeedFactory:** These are concrete classes that implement the FeedFactory class. They return specific feed types.
 - Dairy: Corn (carb), Soybean (protein)
 - Beef: Wheat (carb), Canola (protein)
- **Veterinarian / MinistryVisitor:** These are visitor classes that perform certain checks/actions (vaccination, ear tag check) on each cattle.
 - Veterinarian vaccinates in autumn.
 - MinistryInspector checks ID tags in spring.
- **Subject:** Tracks and controls the location of cattle. Informs the farmer (Observer) if the cattle leave the farm.

- **CattleVisitor (Interface):**Used to define specific treatments for different subspecies of cattle (Dairy Cattle, Beef Cattle).
- **Farmer (Observer):** Gets notified when a cow leaves the farm border.
 - update(Location location): Triggered when a cattle goes out of bounds.
- **BluetoothToZigbeeAdapter:** Converts Bluetooth signal to Zigbee to ensure compatibility with the server.
- **CommunicationDevice :**It is the general interface that defines the communication method of the devices.
- **ZigbeeDevice (Adaptee Class):**Represents the device that can communicate with the Zigbee protocol.
- **BluetoothDevice:** It implements the CommunicationDevice interface and communicates using the Bluetooth protocol.
- **LocationDatabase (Singleton):** Stores location data and cattle records. Only one example is available.
- **BuildFeedPlan:** It is a class that produces and manages special feed mixtures (protein and carbohydrate components) for different animal species in a factory pattern.

4.UML Diagrams



5.Conclusion

The Livestock Farm project has used Observer, Adapter, Singleton, Abstract Factory and Visitor patterns together. The responsibilities of each component are clear and easy to understand. Thanks to the patterns used, future updates have been made easier.